8주차

INSERT · UPDATE · DELETE 그리고 지우지 않는 기록

- INSERT — 새 기록 넣기
- UPDATE — 오입력 정정 (WHERE를 빼먹으면?)
- DELETE와 soft delete — 의무기록은 왜 진짜로 지우지 않는가
- 트랜잭션과 ACID — 정정 입력 중 정전이 나면

> **이번 주 대표 질문**
>
> 잘못 입력된 검사값을 어떻게 '정정'해야 하는가?

INSERT — 새 환자 등록

INSERT

```sql
-- 컬럼을 명시하고 순서대로 값 입력 (권장)
INSERT INTO patient (patient_id, sex, birth_date, blood_type)
VALUES ('P051', 'F', '1988-03-15', 'A+');

-- 여러 행 한 번에
INSERT INTO patient (patient_id, sex, birth_date, blood_type)
VALUES ('P052', 'M', '1979-11-02', 'O+'),
       ('P053', 'F', '2001-06-21', 'B+');
```

> 컬럼명을 생략하면 '테이블 정의 순서'에 의존하게 되어, 나중에 ALTER로 컬럼이 추가되는 순간 코드가 깨집니다. **항상 컬럼명을 명시**하세요.

INSERT vs 제약조건 — 7주차의 복습

임상 질문

방금 넣은 환자가 규칙(제약조건)을 어기면?

```
INSERT INTO patient
  (patient_id, sex, birth_date, blood_type)
VALUES ('P001', 'X', '1988-03-15', 'A+');
```

두 가지가 동시에 걸립니다: P001은 **PK 중복**, 'X'는 **CHECK(sex IN ('M','F')) 위반**. 잘 설계된 스키마에서는 나쁜 데이터가 입구에서 튕겨 나갑니다.

UPDATE — 조건에 맞는 행만 수정

임상 질문

오입력 정정: 혈당 검사 L0044의 값을 167에서 176으로 (재검 결과 반영)

```
UPDATE lab
SET result_value = 176
WHERE lab_id = 'L0044';

SELECT lab_id, test_name, result_value
FROM lab WHERE lab_id = 'L0044';
```

lab_id	test_name	result_value
L0044	Glucose	176

총 1행

UPDATE의 생명은 **WHERE**입니다. 이 문장은 정확히 한 행만 바꿉니다. 그런데 만약 WHERE를 빼먹는다면…?

WHERE 없는 UPDATE — 전 행이 바뀐다

주의

임상 질문

악몽 시나리오: WHERE 없는 UPDATE

```
UPDATE lab
SET result_value = 176;

SELECT COUNT(*) AS 같은값이된행수
FROM lab WHERE result_value = 176;
```

같은값이된행수
93

총 1행

검사 93건 전체가 176이 되었습니다(이 슬라이드는 사본 DB에서 실행한 결과입니다). 실무 수칙: ① **같은 WHERE로 SELECT를 먼저 실행해 대상 확인** → ② **그 다음 UPDATE**.

의무기록은 왜 진짜로 지우지 않는가

핵심

법과 원칙

- 의료법상 진료기록은 **보존 의무**(기록 종류별 보존연한)
- 정정이 필요하면 **수정 이력이 남는 방식**으로
- "누가, 언제, 무엇을, 왜 바꿨나"가 재구성 가능해야
- 의료분쟁 시 기록의 무결성이 곧 신뢰성

기술적 구현

- **Soft delete**: 행을 지우는 대신 `deleted_at` 시각만 기록
- **감사 로그(audit log)**: 변경 전후 값·수행자·시각을 별도 테이블에 자동 적재
- DELETE 권한 자체를 극소수에게만 (14주차 권한 관리)

> 그래서 병원 DB에서 DELETE는 '기록 말소'가 아니라 **'화면에서 숨김' + '이력 보존'**으로 구현됩니다.

soft delete + 감사로그 — 정정의 올바른 모양

실무 패턴

```sql
-- (1) 검사 정정을 이력으로 남기기
CREATE TABLE lab_audit (
  audit_id   INTEGER PRIMARY KEY,
  lab_id     TEXT, old_value REAL, new_value REAL,
  changed_by TEXT, changed_at DATETIME,
  reason     TEXT                       -- '검체 재검으로 정정'
);

-- (2) 삭제 대신 표시만
ALTER TABLE lab ADD COLUMN deleted_at DATETIME;
UPDATE lab SET deleted_at = '2026-04-02 09:30'
WHERE lab_id = 'L0093';   -- DELETE FROM lab ... 대신!
```

이후 모든 조회에 `WHERE deleted_at IS NULL`을 붙이면 '지워진 것처럼' 보이지만 기록은 남습니다. 실제 EMR의 정정·보존 구조는 이보다 정교합니다 — 원리를 보여주는 교육용 축약 예시입니다.

정정 입력 중 정전이 나면 — 트랜잭션과 ACID

시나리오 — 검사값 정정은 '값 수정 + 이력 기록' 두 문장이 한 묶음:

```
BEGIN TRANSACTION;
UPDATE lab SET result_value = 176
  WHERE lab_id = 'L0044';     -- ① 값 정정
INSERT INTO lab_audit
  (lab_id, old_value, new_value,
   changed_by, changed_at, reason)
VALUES ('L0044', 167, 176, 'S001',
   '2026-04-02 09:30', '재검 반영');  -- ② 이력
COMMIT;
```

두 문장 **사이에서 정전**되면? — 값은 바뀌었는데 이력이 없는, 감사 추적이 끊긴 상태.

트랜잭션은 "둘 다 되거나, 둘 다 안 되거나"를 보장합니다. COMMIT 전에 장애가 나면 전체가 자동 취소(ROLLBACK)됩니다.

ACID — 한 장 요약 (투약 사고의 언어로)

특성	약속	병원 버전
Atomicity 원자성	전부 되거나 전부 안 되거나	정정+이력이 반쪽만 실행되는 일은 없다
Consistency 일관성	규칙을 어긴 상태로 끝나지 않는다	제약조건 위반 상태로 저장 불가
Isolation 독립성	동시 작업이 서로를 침범하지 않는다	외래·병동이 동시에 수정해도 뒤섞이지 않음
Durability 영속성	COMMIT된 것은 장애에도 살아남는다	'저장 완료'된 처방은 정전에도 사라지지 않음

ROLLBACK은 수동 취소도 가능: 대량 UPDATE 전에 `BEGIN`으로 열고, 결과를 확인한 뒤 이상하면 `ROLLBACK` — 실습 노트북에서 직접 해봅니다.

연습문제

실습

- **Q1.** 가상 환자 P051을 INSERT하고, visit에 외래 내원 한 건을 연결해 보라
- **Q2.** P051의 혈액형을 UPDATE로 정정하라 (WHERE 확인 습관 지키기)
- **Q3.** WHERE 없는 UPDATE를 (사본에서) 실행해 보고 피해 규모를 확인하라
- **Q4.** BEGIN → UPDATE → ROLLBACK으로 '없던 일로 만들기'를 실습하라
- **Q5.** lab_audit 테이블에 정정 이력 한 건을 직접 남겨 보라

다음 주는 중간고사(1~8주차) — 범위 요약본이 노트북에 포함되어 있습니다.

이번 주 용어 정리

용어	뜻
DML (Data Manipulation Language)	데이터를 다루는 SQL — SELECT·INSERT·UPDATE·DELETE
트랜잭션(transaction)	전부 되거나 전부 안 되어야 하는 작업 묶음 (BEGIN ~ COMMIT)
COMMIT / ROLLBACK	트랜잭션 확정 / 전체 취소
ACID	원자성·일관성·독립성·영속성 — 트랜잭션이 지키는 네 가지 약속
soft delete	행을 지우는 대신 삭제 시각만 표시 — 조회에서만 숨김
감사로그(audit log)	변경 전후 값·수행자·시각·사유를 남기는 이력 테이블

8주차 정리

- INSERT는 컬럼명 명시, UPDATE·DELETE는 **같은 WHERE로 SELECT 먼저**
- 의무기록은 지우지 않는다 — **soft delete + 감사로그**가 법적 보존 의무의 구현 방식
- 트랜잭션: 묶음 작업은 **전부 아니면 전무** (BEGIN·COMMIT·ROLLBACK)
- ACID 네 글자면 '정전 나면 처방은 어떻게 되나요?'에 답할 수 있다

다음 주 9주차 중간고사 → 10주차 모델링과 정규화 ("왜 처방 테이블에 환자 이름이 없을까?")